Project Overview

Python for Data Science | M1 AMS

Project	Topic	Main focus
P1	Ames Housing: House Price Prediction	Regression, data preparation, feature engineering
P2	Phishing Website Detection (proposed)	Classical classification, feature selection, model evaluation
P3	Brain Tumor: MRI Classification	CNN from scratch, transfer learning
P4	EuroSAT: Satellite Image Classification	Computer vision, image representations, model comparison
P5	ECG Anomaly Detection	PCA, autoencoders, reconstruction-based detection
P6	Vanishing and Exploding Signals	Variance propagation, weight initialization, gradient dynamics
P7	Universal Approximation by Shallow Networks	Approximation theory, network width, experimental analysis

GRADED GROUP PROJECT — PROJECT 1

Ames Housing: House Price Prediction

Regression | Main emphasis: EDA, preprocessing, and feature engineering

---

The challenge. A real-estate analyst wants a reasonable estimate of a property's selling price based on its characteristics. Your goal is to build and assess a reproducible approach for predicting residential sale prices, while explaining which information is useful and which data decisions matter.

Dataset

Ames Housing / House Prices – Advanced Regression Techniques (Kaggle). The Kaggle `train.csv` file contains about 1,460 house sales and 79 characteristics (also called *features*), such as living area, neighborhood and building condition. The price to predict is `SalePrice`. Kaggle also supplies a `test.csv` file, but without the actual sale prices. **For this project, use `train.csv` and divide it into your own training and test sets** so you can evaluate your predictions.

Download dataset (Kaggle)

What we expect you to investigate

- **Explore the data:** study sale-price distributions, useful characteristics, missing-value patterns, categorical variables, unusual observations, and relevant relationships.
- **Prepare meaningful predictors:** decide how to handle missing and categorical values; justify transformations and feature choices rather than blindly applying the same rule to every column.
- **Build and compare regression approaches:** begin with a simple approach, then investigate a small number of appropriate methods or feature sets.
- **Evaluate carefully:** choose relevant error metrics, use a sound validation strategy, and study cases where predictions are particularly inaccurate.

What your results should demonstrate

Explain how data preparation affects the predictions, which improvements are supported by experiments, and what the model cannot reliably infer. Include interpretable figures or comparisons that support your conclusions.

Possible directions (not compulsory)

Linear or regularized regression; decision-tree-based methods; alternative encodings; feature selection; target transformations; comparisons across price ranges. **No particular algorithm is required.**

> **Important.** An empty entry does not always mean an unknown value: for example, a missing `PoolQC` value can simply mean that the house has no pool. Check the dataset description before filling in or removing missing values. Keep the test set separate while developing your model so that the final evaluation is fair.

GRADED GROUP PROJECT — PROJECT 2

Phishing Website Detection

Classification | Main emphasis: classical machine learning, feature selection, and evaluation

---

The challenge. Phishing websites imitate legitimate websites to steal sensitive information. Your goal is to develop and evaluate classification methods that distinguish fraudulent websites from legitimate ones, using characteristics of their URLs and web pages. You will investigate which features are most informative, how different classifiers compare, and the consequences of incorrect predictions.

Dataset

Phishing Websites (UCI Machine Learning Repository). The dataset contains 11,055 websites described by 30 precomputed features related to URL structure, domain information, and webpage characteristics. Each website is labeled as legitimate or phishing. The dataset is relatively balanced, with approximately 56% legitimate websites and 44% phishing websites.

Download dataset (UCI)

What we expect you to investigate

- **Explore the data:** examine the class distribution, feature values, correlations, and differences between legitimate and phishing websites.
- **Prepare the data:** establish appropriate training, validation, and test sets, and justify any preprocessing choices.
- **Compare classification approaches:** implement and evaluate several classical machine learning methods, starting with a simple model.
- **Investigate feature selection:** identify informative features and study how reducing or modifying the feature set affects classification performance.
- **Analyze prediction errors:** compare models using suitable metrics, examine false positives and false negatives, and investigate how decision thresholds affect the results.

What your results should demonstrate

Explain which classification approaches perform well and why, how feature selection influences the results, and what trade-offs arise between detecting phishing websites and incorrectly flagging legitimate ones. Support your conclusions with reproducible experiments and appropriate visualizations.

Possible directions (not compulsory)

Logistic regression, decision trees, Random Forest, SVMs, gradient boosting, feature importance, feature selection methods, decision-threshold adjustment, or model interpretability.

> **Important.** The dataset already contains extracted website features; you are not expected to collect websites or implement a web scraper. Keep the test set separate during model development. As the dataset is historical, good results do not necessarily imply equally good detection of modern phishing websites.

GRADED GROUP PROJECT — PROJECT 3

Brain Tumor: MRI Classification

Medical image classification | Main emphasis: image preparation, model comparison, and critical evaluation

---

The challenge. Can a model distinguish different types of brain tumors, and MRI scans without a tumor, from brain images? Your goal is to build and evaluate an image-classification approach, compare meaningful modeling choices, and understand which cases are difficult to classify. The aim is to study an educational benchmark, not to develop a clinical diagnostic tool.

Dataset

Brain Tumor MRI Dataset (Masoud Nickparvar, Kaggle). This widely used collection contains approximately 7,000 MRI images grouped into four categories: *glioma*, *meningioma*, *pituitary tumor*, and *no tumor*. The images originate from multiple public sources and are organized into training and testing folders. Image dimensions and characteristics may vary.

Download dataset (Kaggle)

What we expect you to investigate

- **Explore and prepare the MRI images:** examine class distributions, visualize representative images, and apply appropriate preprocessing and transformations.
- **Train a CNN from scratch:** design and train a small convolutional neural network to classify the MRI images.
- **Apply transfer learning:** use a pretrained model (e.g., ResNet18), replace its classification layer, and train it on the MRI dataset while keeping the pretrained layers frozen.
- **Compare the approaches:** evaluate both models using the same data splits and appropriate metrics. Compare their performance, training time, convergence, and classification errors.

What your results should demonstrate

Explain what transfer learning brings compared with training a CNN from scratch. Support your conclusions with experimental results, discuss the advantages and limitations of each approach, and identify which tumor categories are most difficult to classify.

Possible directions (not compulsory)

Fine-tuning some pretrained layers; comparing training strategies; data augmentation; analyzing confusion matrices and misclassified images.

What your results should demonstrate

Explain which preprocessing and modeling choices matter, where the classifier makes mistakes, and whether observed improvements are supported by a fair comparison. Distinguish performance on this image collection from generalization to new patients or hospitals.

Possible directions (not compulsory)

Simple image descriptors with a standard classifier; a small CNN in PyTorch; data augmentation; transfer learning if feasible; comparison of image resolutions. No particular architecture is required.

> **Important.** Images come from different sources, and patient identifiers are not provided for a reliable patient-level split. Inspect possible overlap between splits, avoid tuning on the test set, and do not interpret a high accuracy as proof of clinical usefulness.

GRADED GROUP PROJECT — PROJECT 4

EuroSAT: Satellite Image Classification

Computer vision / classification | Main emphasis: image representations and experimentation

---

The challenge. Can we recognize types of land use and land cover from satellite images? Your task is to distinguish categories such as forests, residential areas, rivers and agricultural land, and to investigate which image representations and classification approaches work well.

Dataset

EuroSAT (RGB version). A well-known remote-sensing benchmark containing 27,000 labeled 64 × 64 RGB images, divided into ten classes. Use the RGB version (ordinary three-color images), available at the link below, or a documented dataset loader. The 13-band multispectral version is not expected for this course.

Download dataset (EuroSAT repository)

What we expect you to investigate

- **Explore the image data:** visualize examples across classes, inspect label frequencies and image properties, and identify categories that may be difficult to distinguish.
- **Set up a reproducible image pipeline:** document loading, input representation, any normalization or preprocessing, and a suitable train/validation/test separation.
- **Develop and evaluate classification approaches:** begin with a simple classification method and compare it with at least one other meaningful approach; explain the choices and results.
- **Examine errors:** provide class-level results and examples of confusing or incorrectly predicted images, with a discussion of limitations.

What your results should demonstrate

Compare the value of your representation and modeling choices, not just final accuracy. You may use a carefully chosen subset if you justify it and apply the same evaluation protocol across experiments.

Possible directions (not compulsory)

Simple engineered image features with a standard classifier; dimensionality reduction; classical image-processing methods; a small CNN in PyTorch; transfer learning where practical. A CNN and any particular network architecture are optional.

> **Timing and prior experience.** Image processing is taught in W11 and neural networks/CNNs in W12. For this reason, a team with someone already familiar with image data may find it easier to start independently in W8–W10. This is a recommendation, not an entry requirement.

GRADED GROUP PROJECT — PROJECT 5

ECG Anomaly Detection

Anomaly detection | Main emphasis: reconstruction-based methods and experimental comparison

---

The challenge. Electrocardiograms (ECGs) record the electrical activity of the heart. Detecting abnormal heartbeats is an important task in medical signal analysis. Your goal is to investigate whether reconstruction-based methods can distinguish normal from abnormal heartbeats, and whether more complex models lead to better anomaly detection.

Dataset

ECG5000 (UCR Time Series Classification Archive). The dataset contains 5,000 ECG heartbeats, each represented by a sequence of 140 values. The heartbeats belong to five classes: one normal class and four abnormal classes. The dataset provides predefined training and test sets. For this project, treat the task as binary anomaly detection (normal vs. abnormal), while retaining the original labels for further analysis.

Download dataset (ECG5000)

What we expect you to investigate

- **Explore the ECG data:** visualize representative normal and abnormal heartbeats, examine the class distribution, and investigate differences in signal morphology.
- **PCA-based reconstruction:** use Principal Component Analysis to reconstruct normal heartbeats and detect anomalies based on reconstruction errors.
- **MLP Autoencoder:** implement and train a fully connected autoencoder on normal heartbeats. Investigate how the architecture and latent dimension influence reconstruction and anomaly detection.
- **1D Convolutional Autoencoder:** implement an autoencoder using one-dimensional convolutions to capture local patterns in ECG signals.
- **Evaluate and compare:** select anomaly thresholds using validation data, then compare the three approaches using appropriate metrics (e.g., F1-score, balanced accuracy, precision, recall, and PR-AUC). Analyze detection errors across the different abnormal classes.

What your results should demonstrate

Compare PCA, MLP autoencoders, and convolutional autoencoders to investigate whether increasing model complexity improves ECG anomaly detection. Explain how reconstruction errors relate to anomaly scores, which types of heartbeats are difficult to detect, and what the experimental results reveal about the strengths and limitations of each approach.

Possible directions (not compulsory)

Different latent dimensions; reconstruction loss functions; anomaly threshold selection strategies; sensitivity to model initialization; analysis of reconstruction errors and incorrectly classified heartbeats.

> **Important.** Train the reconstruction models using only normal heartbeats. Use validation data to select hyperparameters and anomaly thresholds, keeping the test set untouched until the final evaluation. More complex architectures are not guaranteed to perform better. ECG5000 comes from a single patient, so results should not be interpreted as evidence of generalization to new patients.

GRADED GROUP PROJECT — PROJECT 6

Vanishing and Exploding Signals: A Study of Variance Propagation and Weight Initialization

Theoretical study | Main emphasis on gradient dynamics, variance propagation, and initialization

---

The challenge. A feedforward network of depth L sends an input $x = h^{(0)}$ through the layers

$$h^{(l)} = \sigma\Big(W^{(l)}h^{(l-1)} + b^{(l)}\Big), \qquad l = 1, \dots, L,$$

where $W^{(l)}$ and $b^{(l)}$ are the weights and biases of layer l and σ is the activation. At the start of training the weights are random, so each activation $h^{(l)}$ is a random variable, and one can follow its variance $\mathrm{Var}\big(h^{(l)}\big)$ as it moves up the layers, together with the variance of the gradient $\partial\mathcal{L}/\partial h^{(l)}$ that flows back down during backpropagation, where $\mathcal{L}$ is a loss function. When either variance shrinks or grows geometrically with depth, the gradients vanish or explode and the network barely learns. The scale of the initial weights and the choice of activation σ decide which of these happens. Your goal is to make this concrete. You measure these variances layer by layer, work out the condition on the initial weight scale that keeps them stable across depth, and connect that condition to whether the network actually trains.

Training Data:

MNIST. You train small feedforward networks on MNIST, the standard set of handwritten digits, which is easy to load and quick to run. MNIST gives you a real training signal so you can connect what you measure at initialization to whether the network actually learns. The quantities you record are the variance of the activations and of the back-propagated gradients at each layer.

What we expect you to do:

1. **Build the testbed.** Set up a deep fully connected network in which the activation, the depth, the width, and the initial weight scale are all things you can vary, so you can probe their effect.
2. **Observe the signal.** Find a way to measure how the activations and the back-propagated gradients behave across the layers at initialization, and investigate how this behaviour depends on the activation and on the initial weight scale.
3. **Explain what you see.** Account for the behaviour mathematically, by reasoning about how the signal changes as it passes from one layer to the next, and compare your explanation against the initialization rules of [3] and [4].
4. **Connect to learning.** Investigate how the behaviour you observed at initialization relates to whether, and how fast, each configuration actually trains on MNIST.

What we expect in your report:

A complete study that explains your setup clearly, which network depths and widths you used, how you measured the activation and gradient variance, and how you trained each configuration so the comparison is fair. Present your experiments, describe what you observe as the activation and the initial scale change, and support every claim with evidence from more than one run. Discuss honestly the limits you hit and what you think causes them.

Possible directions (not compulsory)

How deep a network you can still train before it stops learning, the role the activation's derivative plays in the backward pass, a comparison of Glorot against He [4], and whether techniques such as batch normalization change the picture.

Important. Think carefully about *when* you measure, since the picture at initialization and the picture during training need not be the same, so treat them separately. Remember also that the backward pass depends on the activation through its *derivative*, which is worth keeping in mind when you interpret what you see.

GRADED GROUP PROJECT — PROJECT 7

What One Layer Can Do: Universal Approximation of Continuous Functions by Shallow Networks

Theoretical study | Main emphasis on approximation theory, width, and capacity

---

The challenge. A neural network with a single hidden layer, width n and activation σ is the function

$$f_n(x) \;=\; \sum_{k=1}^{n} c_k\, \sigma\Big(a_k^{\top} x + b_k\Big) + c_0,$$

where each neuron shifts and scales the input through its weight a_k and bias b_k, and the output weights c_k combine them. A classical result says that as the width n grows, functions of this form can approximate any continuous target f as closely as you want, so for every tolerance $\varepsilon > 0$ there is a width and a choice of weights with $\max_x |f(x) - f_n(x)| < \varepsilon$ [2]. The catch is that this only guarantees such a network exists, it never says how large n must be, and that depends on the target, a few neurons for a smooth f and many more for a sharp one. Your goal is to make this concrete. You pick a function you fully control, approximate it with single-hidden-layer networks of growing width n, and study how the error shrinks as n increases, including how the gains slow down and how a rougher target needs more width.

Training Data:

Self-generated, no download. You choose the target functions yourself, which is the whole point, because you can measure the error exactly only when you know the true function. A good choice is a smooth one-dimensional function such as a sum of a few sines or Gaussians, and a rougher function for contrast. You create the data by sampling input points, evaluating your function at them, and training a network to reproduce it.

What we expect you to do:

1. **Pick a target.** Define a known continuous function on an interval and sample a set of training points from it. Note that this is a regression task, not classification.
2. **Confirm universality.** Train a network with a single hidden layer and increase its width (number of neurons) until it fits, showing in practice the guarantee proved in [2].
3. **Study the effect of width.** For a range of hidden widths n, investigate how well the network fits the target and how that changes as the width grows.
4. **Vary the target.** Repeat for a smooth and a rougher function, and study the width needed for a good fit.

What we expect in your report:

A complete study that explains your setup clearly, which target functions you chose, how you measured the error, and how you trained each width so the comparison is fair. Present your experiments, describe what you observe as the width changes, and support every claim with evidence from more than one run. Discuss honestly the limits you hit and what you think causes them.

Possible directions (not compulsory)

A comparison of activations, a study of width against depth for the same number of parameters, a look at how the needed width grows as the target becomes rougher, and the idea that error falls with diminishing returns as width grows [1].

> **Important.** Keep two ideas apart. Existence, from [2], says a wide enough single layer can fit the function, while the behaviour of the error as width grows, studied by [1], tells you how the fit improves. When your error stops improving, check whether the width is truly the limit or whether the optimizer simply failed to find the best weights for that width.

References

[1] Andrew R Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information theory*, 39(3):930–945, 1993.

[2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[3] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.